

מדריך MVVM - איך עוברים מקוד ספגטי לקוד של הנדסת תוכנה

“עד היום למדנו איך לגרום לאפליקציה לעבוד. היום נתחיל ללמוד איך לגרום לאפליקציה להיות בנויה נכון. ההבדל הזה הוא ההבדל בין קוד של תלמיד שמצליח לפתור תרגיל, לבין קוד של מפתח שבונה מערכת שאפשר להמשיך לפתח”.

אז מה זה MVVM?

MVVM הוא לא “עוד נושא באנדרואיד”. אנו מדברים על מעבר מחשיבה של “איך לגרום לזה לעבוד עכשיו” לחשיבה של “כיצד בונים מערכת שאפשר להבין, לתקן ולהרחיב גם בעוד חודש”.

עד עכשיו, כשכתבנו אפליקציה- דאגנו להוסיף בכל פעם אקטיביטי, לשלב בתוכו פקדים ולתת לפקדים להפעיל ארועים שונים. ארועים אלו כללו גם לוגיקה נדרשת וגם התנהלות של התצוגה. בקיצור- רוב הלוגיקה והטיפול בתצוגה, היה בתוך האקטיביטי. מדי פעם גם הוספנו מחלקות HELPER/ADAPTER שיסייעו לאקטיביטי אבל בסופו של דבר. האקטיביטי היה מנהל את כל הארוע של האפליקציה.

לעבודה כזו קוראים – כתיבת קוד ספגטי. כלומר מצב שבו כל דבר תלוי בכל דבר. לדוגמה, תלמיד כותב הכל בתוך MainActivity.

- לחיצה על כפתור .
- בדיקת קלט מהמשתמש .
- שליפה מ־ SQLite .
- עדכון TextView .
- פתיחת מסך חדש .
- טיפול בשגיאות .
- שמירת נתונים .
- חישובים לוגיים .
- הצגת הודעות Toast .

בהתחלה זה נראה נוח. הכל נמצא במקום אחד. אבל זו בדיוק המלכודת. כשהאפליקציה קטנה MainActivity, אחת גדולה מרגישה פשוטה. כשהאפליקציה גדלה, היא הופכת למחסן ענק שאין בו מדפים.

התוצאה היא שהתלמיד אומר: "המורה, זה עבד לי אתמול, ועכשיו כשיניתי משהו קטן — הכל נשבר". זה שלב חשוב מאוד. השלב שבו מתחילים ללמד הנדסת תוכנה אמיתית. השלב שבו MVVM נכנס לעבודה.

MODEL VIEW VIEW MODEL

הרציונל

MVVM בא ללמד עיקרון יסודי: **לכל חלק באפליקציה צריך להיות תפקיד ברור.**

מדובר בצורת ארגון הקוד באפליקציה כך שכל חלק יקבל תפקיד ברור. לפי התייעוד הרשמי של Android ViewModel, היא מחלקה שמחזיקה מצב של מסך ולוגיקה **שקשורה למסך**, והיתרון המרכזי שלה הוא שהמידע יכול להישמר גם כאשר המסך נוצר מחדש, למשל- בסיבוב מסך.

במקום שכל הקוד יישב בתוך MainActivity, אנחנו מחלקים את האחריות בין כמה שכבות:

- שכבת מסך- VIEW - כל מה שהמשתמש רואה/ נוגע בו.
- שכבת ViewModel המוח שמנהל את המסך
- שכבת Model הנתונים והאמת של האפליקציה

חלק	משמעות	תפקיד פשוט
Model	מודל / נתונים	מייצג את המידע של האפליקציה
View	תצוגה	המסך שהמשתמש רואה
ViewModel	מנהל מצב המסך	שומר את הנתונים והלוגיקה של המסך

מתי כדאי להשתמש ב־ MVVM ?	מתי MVVM עלול להיות מוגזם?
• מסך עם הרבה נתונים . • טופס עם בדיקות תקינות . • עבודה עם SQLite / Room / Firebase • עבודה עם API . • מספר מצבי מסך : טעינה, הצלחה, שגיאה . • צורך לשמור מידע לאחר סיבוב מסך . • פרויקט לימודי שרוצים שיהיה מסודר לפי מחלקות	במסך פשוט מאוד כמו : • מסך פתיחה עם כפתור אחד . • מסך About . • תרגיל ראשון של Toast. • אפליקציה של 10 שורות קוד . אלו מקרים שעבורם MVVM עלול להיות יותר מדי "טקס" מסביב לקו

א. שכבת מסך - VIEW - כל מה שהמשתמש רואה/ נוגע בו.

ה' View הוא מה שהמשתמש רואה ונוגע בו.

באנדרואיד Java זה בדרך כלל:

- Activity
- Fragment
- קבצי XML וכל הרכיבים שיש בהם

התפקיד של ה' View הוא להציג מידע, לקבל פעולות מהמשתמש ולצפות בשינויים שמגיעים מה .viewmodel.

ה' View לא אמור לחשב חוקים עסקיים עמוקים. הוא לא אמור לדעת איך שולפים נתונים ממסד נתונים. הוא לא אמור לדעת איך פועלת קריאת API . גם שמירת המצב state של המסך - לא צריכה להיות בו. הוא כן יכל לבצע פעולות תצוגה אבל לא את הלוגיקה המרכזית של האפליקציה

ב. שכבת ViewModel המוח שמנהל את המסך. הלב של הMVVM

ה' ViewModel יכול לשמור את מצב המסך בזמן שינויי תצורה, למשל סיבוב מסך. לכן במקרים רבים אין צורך לטעון מחדש רק משום שה' Activity נבנה מחדש . עם זאת, בפתיחה ראשונה של המסך, או לאחר שינוי אמיתי בנתונים, עדיין נטען או נרענן את הנתונים..

הוא עונה על שאלות כמו:

- מה הנתונים שצריך להציג כרגע ?

- האם יש שגיאה ?

- האם צריך להציג טעינה ?

- מה קורה כשהמשתמש לוחץ על כפתור ?

- איזה מידע צריך לשלוח למסד הנתונים או ל' API?

ה' ViewModel לא מצייר כפתורים, או מחזיק TextView, הוא לא מכיר ישירות את העיצוב של המסך. ViewModel לא אמור להיות תלוי במסך. המסך תלוי בו. הוא לא מחזיק REFERENCE לאקטיביטי או לפרגמנט.

ג. שכבת Model - הנתונים של האפליקציה

ה־ Model מייצג את הנתונים ואת מקורות המידע.

לדוגמה :

- מחלקת Student
- מחלקת Task
- מחלקת Recipe
- מחלקת Weather
- מחלקות שמייצגות טבלאות במסד נתונים
- מחלקות שמייצגות תשובה מ־ API
- בפרויקט מתקדם יותר, ה־ Model יכול לכלול גם :
- Repository - לא מופיע אצלינו באפליקציה. מי שממלא תפקיד דומה – יהיה ה Manager
- DAO
- Room
- SQLite Helper
- מחלקת API
- מחלקת Manager שמביאה נתונים

הרעיון הפשוט הוא ה־ Model הוא המקום שבו נמצאים הנתונים והגישה אליהם.

לסיכום המבוא

קוד ספגטי הוא כמו מסעדה שבה אדם אחד עושה הכל: מקבל הזמנות, מבשל, שוטף כלים, גובה כסף ומנקה שולחנות. זה אולי עובד לקוח אחד. אבל כשיש הרבה לקוחות — הכל קורס.

MVVM הוא כמו מסעדה מסודרת:

- המלצר מקבל הזמנות ומגיש אוכל View.

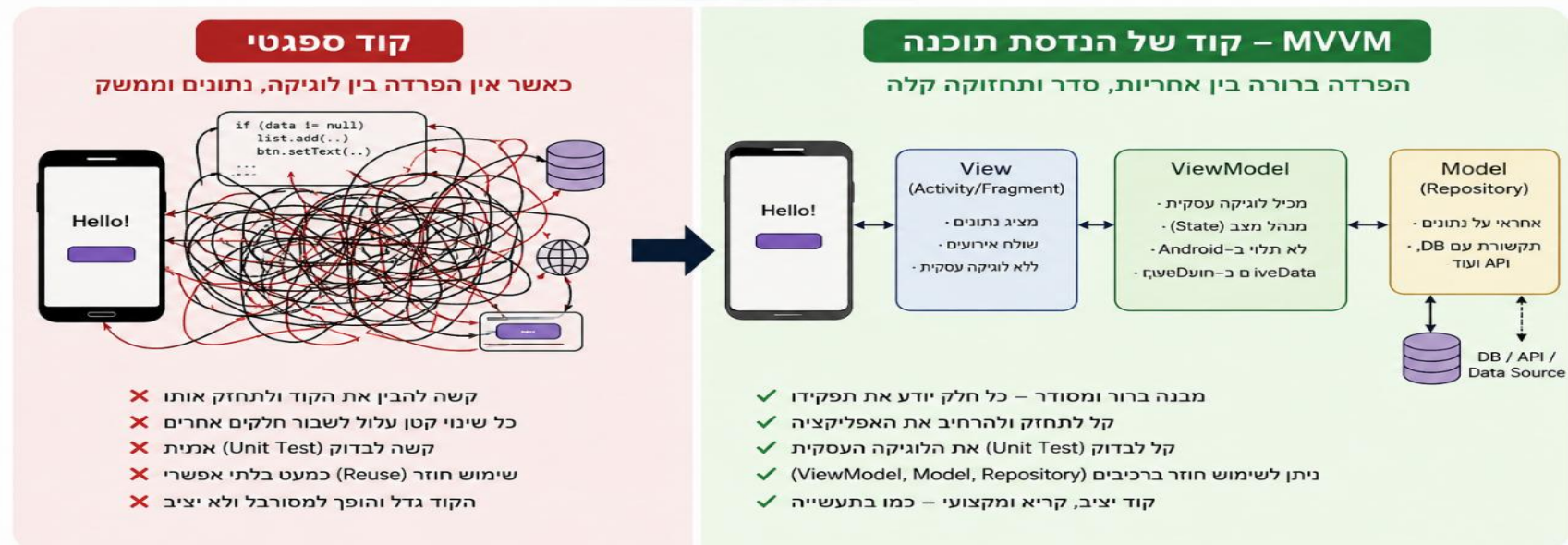
- מנהל המשמרת מחליט מה צריך לקרות ViewModel.

- המטבח והמחסן מחזיקים את האוכל והנתונים Model.

ב-MVVM ה-Activity לא אמור להכיל את הלוגיקה של האפליקציה. תפקידו הוא בניהול התצוגה, לבצע קלט מהמשתמש, לתפעל מאזינים ולהגיב לשינויים שמגיעים מה ViewModel.

MVVM – איך עוברים מקוד ספגטי לקוד של הנדסת תוכנה

למה זה חשוב?



MVVM הוא לא רק דפוס ארכיטקטורה – זו דרך חשיבה שמובילה אותנו מקוד מבולגן לאפליקציות איכותיות, יציבות וקלות לפיתוח – כמו בעולם ההנדסה האמיתית.



דוגמה מפורטת

בטרם אציג דוגמה עם קוד , אדגיש מספר דברים שצריך לשאול כשמפתחים אפליקציה ומפרקים אותה לשכבות שצויינו לעיל.

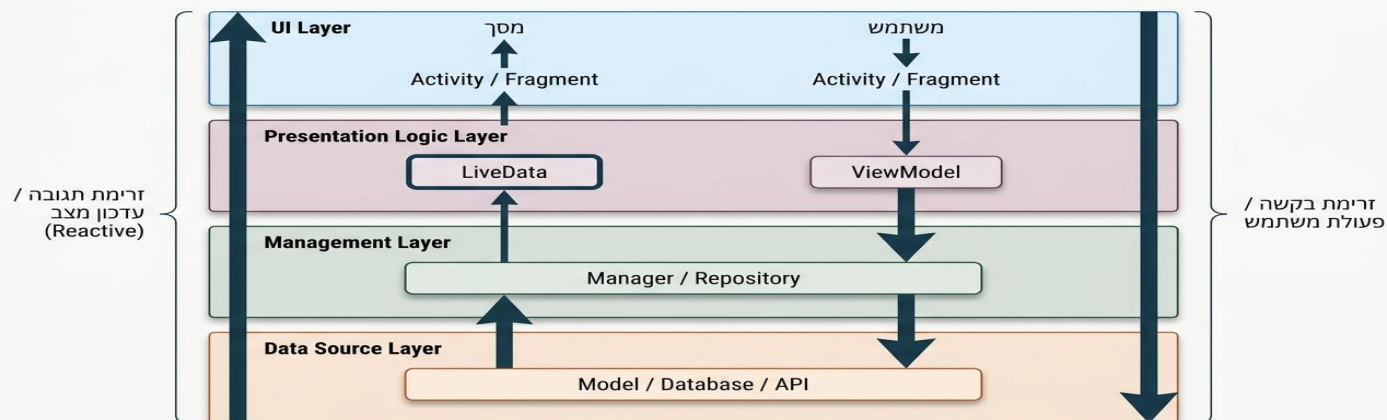
שאלה	למשל האם הקוד עוסק ב...	השכבה אליה שייך	הערות
האם הקוד קשור למסך?	findViewById setText setOnClickListener Toast RecyclerView Adapter Intent	VIEW	
האם הקוד מקבל החלטה?	if (age >= 18) if (password.length() < 6) if (price > 100) if (productList.isEmpty())	ברוב המקרים מדובר בלוגיקה ולכן יכתב בviewmodel	
האם הקוד משנה או שומר נתונים?	list.add(...) list.remove(...) user.setName(...) sharedPreferences.edit() database.insert(...)	מאוד תלוי שינוי רשימה - ViewModel או Manager שמירה במסד נתונים - Repository / Manager שינוי אובייקט - Model או ל Manager הצגת תוצאה - Activity / Fragment	ה־Activity לא אמורה להיות המחסן של הנתונים. באקטיביטי תהיה הפעלה של הVIEWMODEL
האם הקוד צריך להחזיר למסך תוצאה מסויימת	ההמלצה היא ש ViewModel לא יעבוד ישירות מול רכיבי פלט	ViewModel משנה נתון LiveData מודיע שיש שינוי ל Activity שמאזין לו Activity מקבל את השינוי ומעדכן את המסך	צריך לחשוב על אופן החזרת התוצאה
האם הקוד צריך להישאר גם אחרי סיבוב מסך?			אם הרשימה נמצאת בתוך MainActivity, היא עלולה להימחק כאשר המסך נוצר מחדש. אם היא

זיו יהלום	תיכון פארק המדע נס ציונה	יוני 2026
		נמצאת בתוך ViewModel, היא נשמרת טוב יותר בזמן שינויי תצורה כמו סיבוב מסך.
האם הקוד יכול להיבדק בלי מסך?	אם אפשר לבדוק את הקוד בלי לפתוח Activity הוא כנראה שייך ל- ViewModel, Manager או Model.	<pre>public boolean isPasswordValid(String password) { return password.length() >= 6; } textViewMessage.setText("סיסמה" סיסמה"");</pre>
בדיקת תקינות אורך סיסמה-לא צריך מסך.		הצגת תוצאת בדיקת תקינות סיסמה- כן צריך מסך

כללי אצבע

שייך בדרך כלל ל...	אם הקוד...
Activity / Fragment	מוצא רכיב במסך
Activity / Fragment	מאזין ללחיצה
Activity / Fragment	משנה טקסט במסך
ViewModel	בודק אם קלט תקין
ViewModel	מחליט מה לעשות
ViewModel	שומר רשימה זמנית
Model	מייצג אובייקט כמו מוצר / תלמיד / משימה
Manager / Repository	עובד מול SQLite / API / SharedPreferences
ViewModel דרך LiveData	מעדכן נתון שהמסך צריך להציג

מסלול המידע משכבת התצוגה למסד הנתונים ובחזרה



NotebookLM

תפקידי השכבות במסלול הדו-כיווני

השכבה	תפקיד בדרך למטה	תפקיד בדרך למעלה
Activity / Fragment	קליטת פעולות המשתמש (לחיצות, הקלדה) והעברתן ללוגיקה	הצגת נתונים עדכניים למשתמש על המסך
ViewModel & LiveData	קבלת בקשות מהתצוגה (ViewModel)	חשיפת הנתונים באמצעות LiveData לעדכון אוטומטי של התצוגה
Manager / Repository	ניתוב הבקשה למקור הנתונים המתאים	איסוף, ריכוז ועיבוד המידע ממקורות הנתונים
Model / Database / API	קבלת השאילתה	שליפת הנתונים הגולמיים והחזרתם במעלה הערימה

ההפרדה לשכבות מאפשרת קוד נקי, מודולרי ותגובתי (Reactive), כאשר LiveData מגשר בין הנתונים לתצוגה ללא תלות ישירה.

NotebookLM

אדגים על גבי שתי אפליקציות שונות עם הבדלים חשובים ביניהם. האחת Counter שמעלה ומורידה ערך של מונה.

זרימת המידע באפליקציה

המשתמש רואה שלושה כפתורים: הגדל, הקטן, איפוס. בכל לחיצה הערך משתנה, והמסך מציג גם הודעת מצב: המספר חיובי, המספר שלילי, או המספר הוא אפס.

- בלחיצה על כפתור הגדל- האקטיביטי מזהה את הלחיצה
 - האקטיביטי שולח בקשה ל CounterViewModel .
 - CounterViewModel פונה ל-CounterManager, שמעדכן את ערך המונה. לאחר מכן CounterViewModel קורא את הערך החדש, קובע את הודעת הסטטוס המתאימה, ומעדכן את ה-LiveData.
- כלומר המשתמש מבצע פעולה, הנתון משתנה, המסך מתעדכן.

התצוגה שהמשתמש רואה VIEW מיוצג כאן על ידי ה MAINACTIVITY

מי שמבצע את החשיבה מחליט ומנהל את זרימת הנתונים VIEWMODEL שמיוצג כאן על ידי CounterViewModel

מי ששומר את הנתון ומבצע את הפעולות על הנתון- MODEL שמיוצג כאן על ידי CounterManager

שני כלים חשובים שצריך לתת עליהם את הדעת

האחד- **Observer**. הוא נמצא באקטיביטי או בפרגמנט. תפקידו להאזין לשינויים. הוא נמצא בקשר רציף עם הכלי השני שנקרא **LiveData**. כלי זה הוא הגשר שבין שכבת הלוגיקה לבין שכבת התצוגה.

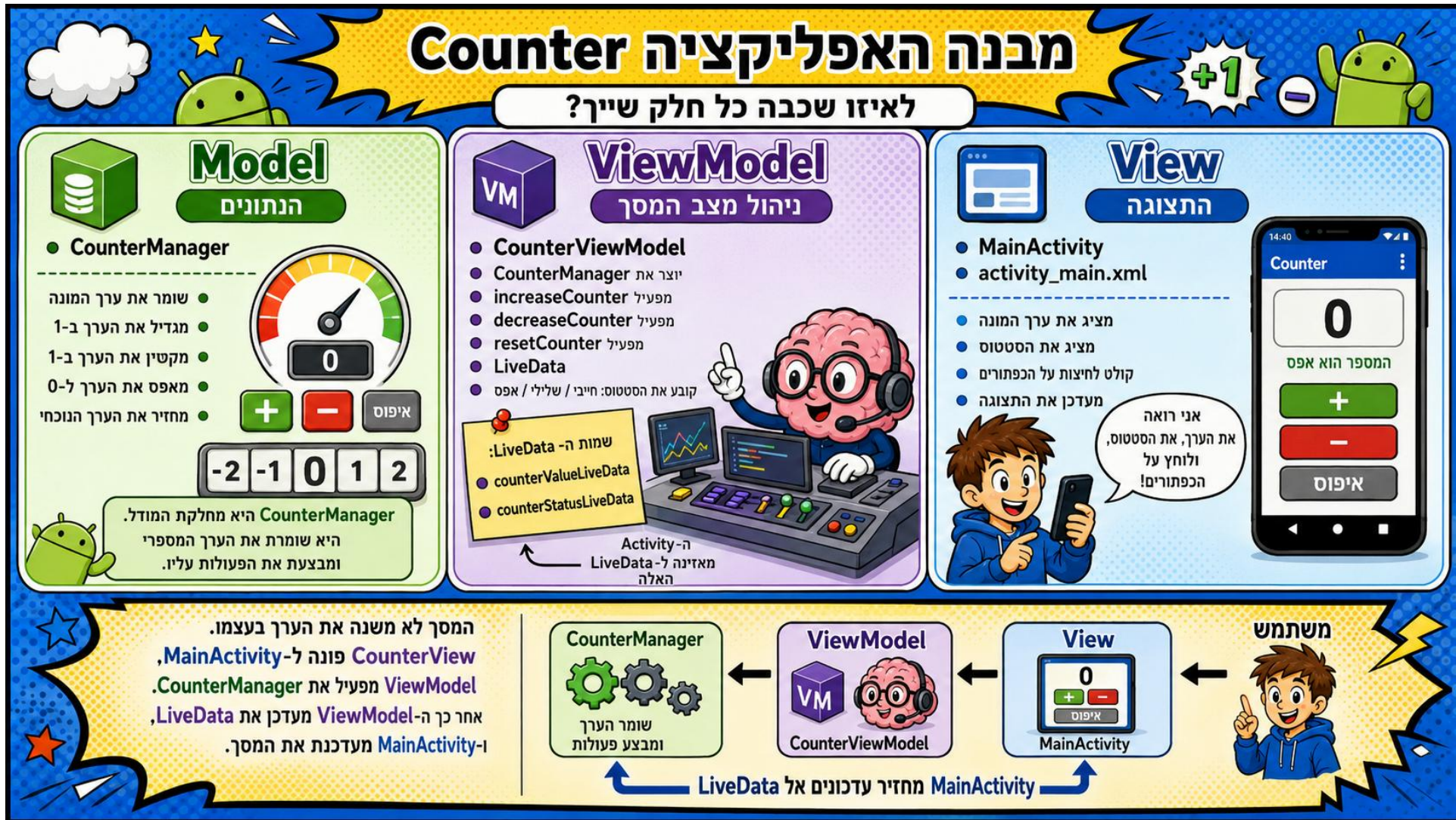
כלומר

Observer מאזין ל-LiveData

LiveData מקשר בין ה-ViewModel לבין ה-Activity

Manager מקשר בין ה-ViewModel לבין הנתונים / Database / Model

קוד מלא של האפליקציה- בנספח ראשון.



אפליקציה שניה היא אפליקציה של קניות.

באפליקציה זו בחרנו להשתמש ב־AndroidViewModel משום שמחלקת ניהול הנתונים זקוקה ל־Context כדי לעבוד עם SQLite.

חשוב להדגיש. לא כל אפליקציה עם מסד נתונים חייבת AndroidViewModel. משתמשים בזה כאשר ה־ViewModel עצמו צריך לקבל Application Context או ליצור מחלקה שתלויה בו.

בניגוד לאפליקציה הראשונה שבה עבדנו ללא מסד נתונים, באפליקציה זו אנו מתחילים לעבוד עם שכבת נתונים אמיתית, ה־ViewModel כבר לא עוסק רק בלחיצות ובחישוב קטן, אלא מתחיל לתווך בין המסך לבין תשתית שלמה של מידע. וכאשר שכבת המידע נשענת על רכיב של אנדרואיד כמו SQLite, נדרש לעיתים חיבור מסודר אל סביבת האפליקציה. האפליקציה מפסיקה לעבוד רק עם "ערכים בזיכרון", ומתחילה לעבוד עם שכבת מידע קבועה ומתמשכת.

האפליקציה צריכה :

- לפתוח גישה למסד הנתונים.
- לקרוא נתונים ששמורים במכשיר.
- לעדכן נתונים קיימים.
- למחוק נתונים.
- לשמור מבנה קבוע של טבלאות.

כלומר, הלוגיקה של האפליקציה פועלת אחרת. היא נשענת על מערכת מקומית של נתונים. ברגע שיש מסד נתונים, ה־ViewModel לא רק "מגיב למסך", אלא הופך לנקודת תיווך בין שתי מערכות שונות :

מערכת אחת : המסך והאינטראקציה עם המשתמש

מערכת שניה : שכבת נתונים מקומית, אמיתית, קבועה, ומבוססת מערכת. כאשר שכבת הנתונים הזאת תלויה ביכולות של אנדרואיד, ה־ViewModel צריך דרך בטוחה ומסודרת להתחבר אליה. דרך זו מתבצעת בצורה פשוטה בעזרת AndroidViewModel. לא בכל מקרה זה יהיה כך. במבנים מתקדמים יותר אפשר להשתמש ב־ViewModel רגיל, ולהעביר את ה־Context רק לשכבת הנתונים או ל־Manager.

מבנה האפליקציה ShoppingMVVM

לאיזו שכבה כל חלק שייך?



Model

הנתונים

- ShoppingItem
- ShoppingManager
- SQLite

מיוצג ShoppingItem
מנהלת ShoppingManager
הנתונים ועובדת מול SQLite



ViewModel

ניהול מצב המסך

- ShoppingViewModel
- בודק קלט
- מפעיל הוספה ומחיקה
- טוען מחדש את הרשימה
- LiveData



View

התצוגה

- MainActivity
- activity_main.xml
- item_product.xml
- ShoppingAdapter

מציים את המסך, קולטים לחיצות,
ומעדכנים את התצוגה.



המסך לא פונה ישירות למסד הנתונים. הוא פונה ל-ViewModel,
וה-ViewModel מפעיל את ShoppingManager.

AndroidViewModel

איננו ViewModel "אחר לגמרי." הוא עדיין ViewModel אבל הוא מתאים יותר למצבים שבהם שכבת הלוגיקה צריכה לעבוד מול חלקים של האפליקציה שתלויים ב־Application.

במילים אחרות- AndroidViewModel מתאים יותר כאשר ה־ViewModel צריך להקים או לנהל גישה לשכבת נתונים שמבוססת על תשתית אנדרואיד. למשל:

- SQLite
- SharedPreferences
- גישה למשאבים כלליים של האפליקציה

למה לא לעבוד ישירות עם Activity ?

יכלנו לחשוב שאם צריך Context, פשוט נעביר ל־ViewModel את MainActivity אבל זה פחות נכון כי ה־ViewModel אמור להיות שכבת ביניים יציבה יחסית, ולא להיות קשור ישירות למסך עצמו. אם נחבר אותו ל־Activity, נקרב אותו יותר מדי לשכבת התצוגה ומבחינה תכנונית, זה פחות נקי.

לעומת זאת Application, הוא הקשר כללי יותר, רחב יותר, ויציב יותר. אם כבר צריך Context, עדיף שה־ViewModel יקבל אותו דרך AndroidViewModel.

כאשר אין מסד נתונים כאשר אין מסד נתונים, ה־ViewModel עשוי לעסוק בעיקר בניהול מצב המסך ובלוגיקה פשוטה. כאשר יש SQLite או רכיב אנדרואיד אחר שדורש Context, צריך לתכנן כיצד מחברים את שכבת הנתונים בצורה בטוחה - ובמדרוך זה נעשה זאת בעזרת AndroidViewModel..

המשמעות מבחינת תכנון נכון תתבטא באמצעות 3 עקרונות

- המסך לא יטפל במסד הנתונים- ה־Activity לא אמורה לפתוח SQLite ולבצע שאילתות
- שה־ViewModel לא יהיה תלוי במסך - הוא לא אמור להישען על MainActivity
- שהגישה לשכבת הנתונים תהיה מסודרת ויציבה - ולכן נותנים ל־ViewModel גישה ל־Application, לא ל־Activity

לסיכום-עם מסד נתונים, ה־ViewModel כבר לא רק "חושב על המסך" הוא גם מתווך בין המסך לבין שכבת מידע אמיתית של האפליקציה ולכן לעיתים נכון יותר לבסס אותו על AndroidViewModel

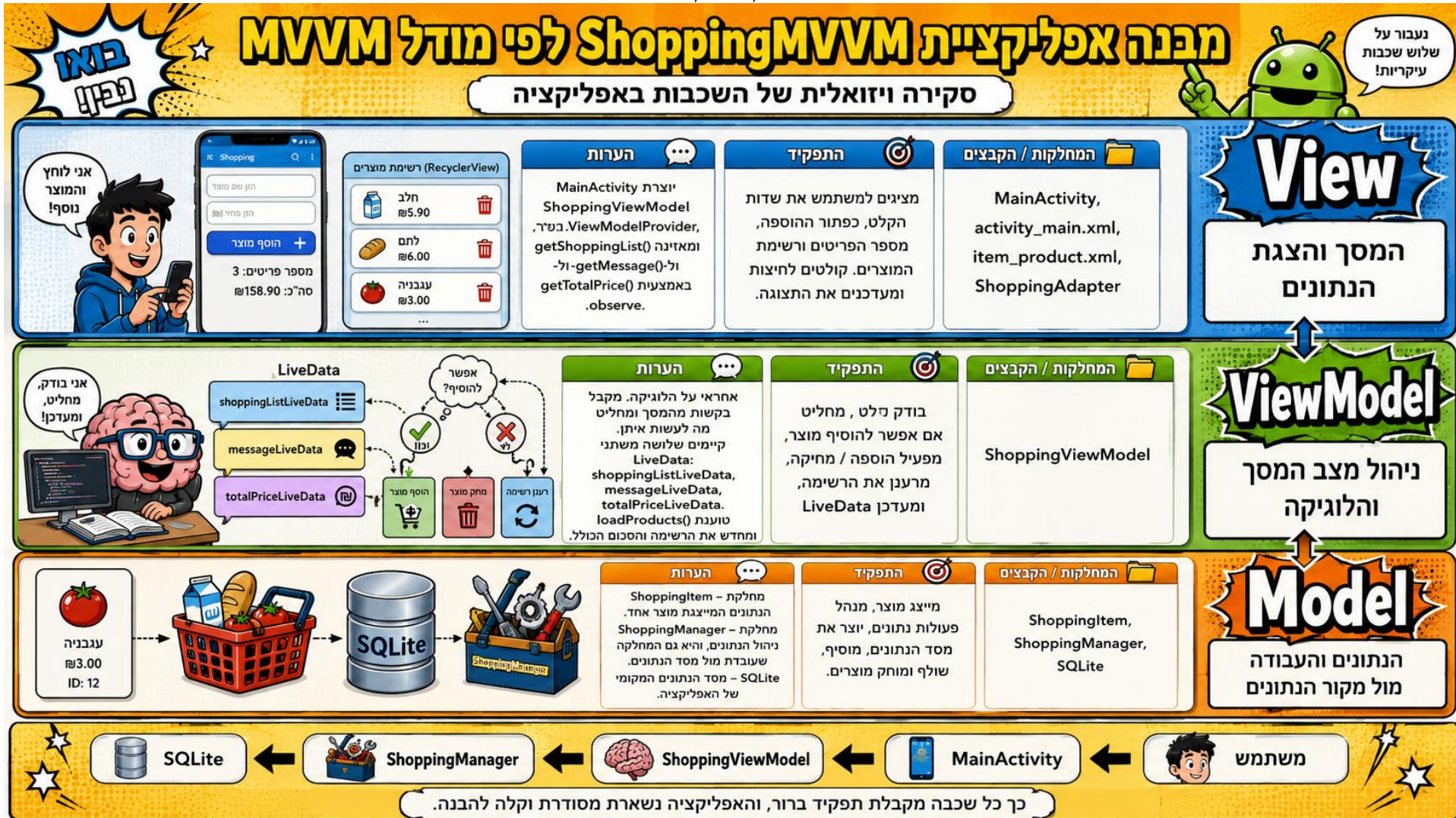
זיו יהלום
אז מה יש לנו באפליקציה ShoppingMVVM

באופן כללי התהליך שהאפליקציה מאפשרת
המשתמש מבצע פעולה הוספה/מחיקה של מוצר
הפעולה נשלחת אל `ShoppingViewModel`.

ה־`ViewModel`

- בודק את הנתונים,
- מפעיל את מחלקת ניהול הנתונים,
- הנתונים נשמרים או נמחקים ממסד הנתונים,
- ואז הרשימה נטענת מחדש והמסך מתעדכן.

שכבה במודל MVVM	הביטוי של זה באפליקציה	המחלקות / הקבצים	התפקיד	הערות
View	המסך והצגת הנתונים	MainActivity, activity_main.xml, item_product.xml, ShoppingAdapter	מציגים למשתמש את שדות הקלט, כפתור ההוספה, מספר הפריטים ורשימת המוצרים. קולטים לחיצות ומעדכנים את התצוגה.	המחלקה יוצרת את ה- ShoppingViewModel בעזרת ViewModelProvider, ומאזינה ל- getShoppingList() ול- getMessage() וגם ל- getTotalPrice() באמצעות observe .
ViewModel	ניהול מצב המסך והלוגיקה	ShoppingViewModel	בודק קלט, מחליט אם אפשר להוסיף מוצר, מפעיל הוספה / מחיקה, טוען מחדש את הרשימה, ומעדכן LiveData.loadProducts(). מופעלת כאשר צריך לטעון או לרענן את הרשימה. בפתיחת המסך, לאחר הוספה מוצלחת, ולאחר מחיקה.	המוח של האפליקציה. מקבל בקשות מהמסך ומחליט מה לעשות איתם. קיימים שלושה משתני LiveData חשובים. private final MutableLiveData<List<ShoppingItem>> shoppingListLiveData; private final MutableLiveData<String> messageLiveData; totalPriceLiveData - הראשון לשמירת רשימת המוצרים - השני לשמירת הודעות למסך - השלישי לסכום הכולל.
Model	הנתונים והעבודה מול מקור הנתונים	ShoppingItem, ShoppingManager, SQLite	מייצג מוצר, מנהל פעולות נתונים, יוצר את מסד הנתונים, מוסיף, שולף ומוחק מוצרים.	ShoppingItem – מחלקת הנתונים המייצגת מוצר אחד ShoppingManager – ניהול הנתונים- מחלקה שעובדת מול מסד הנתונים.





שלבים במחיקת מוצר

תהליך מחיקת מוצר באפליקציית ShoppingMVVM

אני מוחקת מוצר מהרשימה!

RecyclerView

- חלב 8.50 ₪
- לחם 6.00 ₪
- בננות 3.00 ₪

LiveData

- shoppingListLiveData (רשימת מוצרים)
- totalPriceLiveData (סכום כולל)
- messageLiveData (הודעה למשתמש)

הכול מתעדכן אוטומטית!

שלב	מה קורה?	מי אחראי?
1	המשתמש לוחץ על כפתור מחיקה בשורה של מוצר	המשתמש
2	ShoppingAdapter קולט את הלחיצה	ShoppingAdapter
3	ה-Adapter מעביר את המוצר למחיקה ל-MainActivity דרך המאזין	ShoppingAdapter
4	MainActivity קוראת ל-shoppingViewModel.deleteProduct(shoppingItem)	MainActivity
5	ShoppingViewModel מבקש מ-ShoppingManager למחוק את המוצר	ShoppingViewModel
6	ShoppingManager מוחק את הרשומה המתאימה מתוך SQLite לפי id	ShoppingManager
7	ShoppingViewModel מפעיל loadProducts()	ShoppingViewModel
8	totalPriceLiveData מתעדכן מחדש	ShoppingViewModel
9	messageLiveData מתעדכן בהודעת מחיקה	ShoppingViewModel
10	MainActivity מאזינה ל-LiveData ומעדכנת את הרשימה, מספר הפריטים, הסכום הכולל וההודעה למשתמש	MainActivity

SQLite

נמחק! מוצר

מי מאזין למי?

מי מאזין למי באפליקציה?

מפת ההאזנה ב-ShoppingMVVM

העיקרון החשוב

המסך מאזין למשתמש

המסך מאזין גם ל-LiveData

ה-ViewModel לא מאזין לכפתורים

מסד הנתונים לא מעדכן את המסך לבד

מי?	למה הוא מאזין?	האם הוא מאזין?	הסבר פשוט
1 MainActivity	למסך	כן	היא מאזינה ללחיצה על כפתור "הוסף מוצר". כאשר המשתמש לוחץ, היא קוראת <code>shoppingViewModel.addProduct(...)</code> .
2 ShoppingAdapter	ללחיצה על כפתור בחינה בתוך שורה ברשימה	כן	בכל שורה יש כפתור מחיקה. כאשר לוחצים עליו, ה-Adapter מעביר את המוצר למחיקה <code>onDeleteClick</code> .
3 MainActivity	ל-LiveData	כן	MainActivity מאזינה ל- <code>shoppingListLiveData</code> , ל- <code>totalPriceLiveData</code> ול- <code>messageLiveData</code> . כאשר מעדכנת את מספר הפריטים, כאשר הסכום משתנה, היא מעדכנת את הסכום הכולל. כאשר הודעה משתנה, היא מציגה <code>Toast</code> .
4 ShoppingViewModel	לא מאזין למסך	לא	הוא לא מאזין לכפתורים. הוא מקבל קריאה מה- <code>Activity</code> , למשל <code>addProduct</code> או <code>deleteProduct</code> .
5 ShoppingManager	לא מאזין	לא	ShoppingManager מקבלת פקודות מה- <code>ViewModel</code> ומבצעת את הפעולות מול <code>SQLite</code> . היא לא מאזינה למסך ולא ל-LiveData.
6 SQLite Database	לא מאזין ולא מעדכן את המסך לבד	לא	רק שומר, שולף ומוחק נתונים לפי פקודות שהוא מקבל.

כיוון הזרימה: מימין לשמאל

משתמש: אני לוחץ! / אירועי משתמש (לחיצות, הוספה וכו')

MainActivity: אני מאזין! / מאזין

ShoppingViewModel: אני חושב ומקבל קריאות! / מאזין

ShoppingManager: אני מבצעת! עובד מול המסד! / מאזין

SQLite Database: לא מעדכן את המסך לבד / ערוץ עדכון LiveData

מקרא: מאזין (אוזן), לא מאזין (X)

ב-ShoppingMVVM יש שני סוגי האזנה עיקריים:

האזנה לפעולות המשתמש, והאזנה לשינויים ב-LiveData. כך כל שכבה נשאר מסודרת וברורה.



סיכום



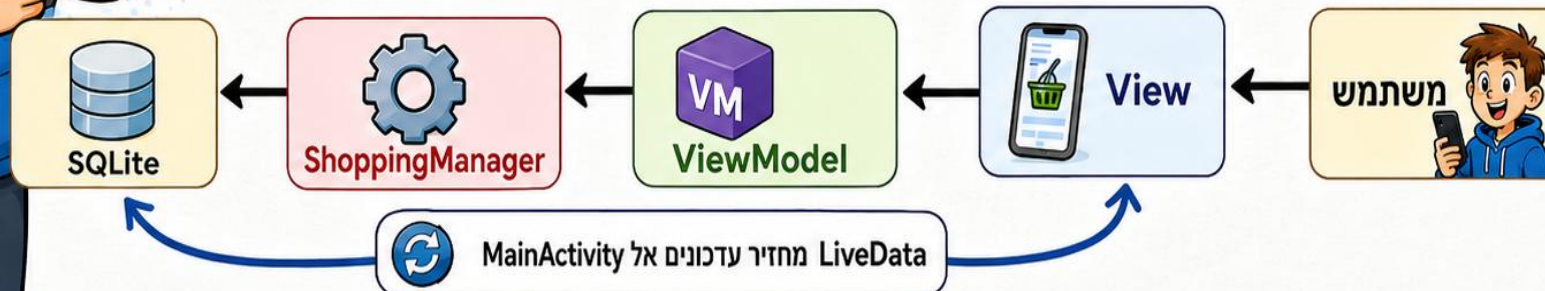
מה חשוב לזכור על ShoppingMVVM?



כשמחלקים אחריות בין שכבות,
הקוד ברור יותר,
מסודר יותר
וקל יותר לתחזוקה!



	View 1	.MainActivity, ShoppingAdapter
	ViewModel 2	ShoppingViewModel מנהלת את מצב המסך ואת הלוגיקה.
	Model 3	.ShoppingItem, ShoppingManager ו- SQLite
	האזנה 4	מאזינה ל shoppingListLiveData, ל messageLiveData, ול totalPriceLiveData.
	ShoppingManager 5	ShoppingManager אחראית על כיהול נתונים ועל הענודה מיל SQLite.
	הזרימה 6	הפשתמש פועל במסך. MainActivity שולחת בקשה ל- ShoppingViewModel. ה- ViewModel מפעיל את ShoppingManager. לאחר ונכן LiveData מחעדכן ו- MainActivity מעדנות את המסך.



לזכרו של אבי צבי יהלום-

איש חינוך וערכים- ניצול שואה

יליד בילגוריי שבפולין

אשר הלך לעולמו והוא כבן 90

MAINACTIVITY

```

/**
 * האפליקציה של הראשית Activity-ה.
 * ViewModel-ל (Views) החזותיים הרכיבים קישור, המשתמש ממשק הצגת על אחראית,
 * המשתמש של לחיצה לאירועי והאזנה.
 */
public class MainActivity extends AppCompatActivity {
    /**
     * המונה של המספרי הערך את המציג TextView.
     */
    private TextView textViewCount;

    /**
     * (אפס/שלילי/חיובי) המונה של הסטטוס תיאור את המציג TextView.
     */
    private TextView textViewStatus;

    /**
     * המונה להגדלת כפתור.
     */
    private Button buttonIncrease;

    /**
     * המונה להקטנת כפתור.
     */

```



```
private Button buttonDecrease;
```

```
/**
```

```
 * המונה לאיפוס כפתור.
```

```
 */
```

```
private Button buttonReset;
```

```
/**
```

```
 * המסך של והלוגיקה הנתונים את שמנהל ViewModel-ה.
```

```
 */
```

```
private CounterViewModel counterViewModel;
```

```
/**
```

```
 * Activity-ה של הכניסה נקודת.
```

```
 * והמאזינים ViewModel-ה את ומגדירה Views-ה את מחברת, UI-ה את מאתחלת.
```

```
 * @param savedInstanceState מצב שמור של Activity-ה (קיים אם).
```

```
 */
```

```
@Override
```

```
protected void onCreate(Bundle savedInstanceState) {
```

```
    super.onCreate(savedInstanceState);
```

```
    EdgeToEdge.enable(this);
```

```
    setContentView(R.layout.activity_main);
```

```
    connectViews();
```

```
    setupViewModel();
```

```
    setupListeners();
```

```
}
```

```
/**
```

** המחלקה למשתני Layout-מה UI-ה רכיבי את ומחברת מוצאת **

**/*

```
private void connectViews() {
    textViewCount = findViewById(R.id.textViewCount);
    textViewStatus = findViewById(R.id.textViewStatus);
    buttonIncrease = findViewById(R.id.buttonIncrease);
    buttonDecrease = findViewById(R.id.buttonDecrease);
    buttonReset = findViewById(R.id.buttonReset);
}
```

*/***

** LiveData-ה על Observers-ה את ומגדירה ViewModel-ה את מאתחלת **

** אוטומטית מתעדכן UI-ה, משתנים ViewModel-ב הנתונים כאשר **

**/*

```
private void setupViewModel() {
    counterViewModel = new ViewModelProvider(this).get(CounterViewModel.class);
```

// המונה בערך לשינויים האזנה

```
counterViewModel.getCounterValue().observe(this, count -> {
    textViewCount.setText(String.valueOf(count));
});
```

// המונה בסטטוס לשינויים האזנה

```
counterViewModel.getCounterStatus().observe(this, status -> {
    textViewStatus.setText(status);
}); }
```

*/***

**UI-ב לכפתורים לחיצה מאזיני מגדירה **

**ViewModel-ב המתאימה לפעולה קוראת לחיצה כל **

**/*

```
private void setupListeners() {
    buttonIncrease.setOnClickListener(v -> {
        counterViewModel.increaseCounter();
    });

    buttonDecrease.setOnClickListener(v -> {
        counterViewModel.decreaseCounter();
    });

    buttonReset.setOnClickListener(v -> {
        counterViewModel.resetCounter();
    });
}
```

מחלקת COUNTERVIEWMODEL

*/***

** הראשי המסך של ViewModel-ה מחלקת.*

** LiveData-ב שימוש תוך UI-ה עבור העסקית והלוגיקה הנתונים ניהול על אחראית*

** שלו ובסטטוס המונה בערך שינויים על התצוגה את לעדכן כדי **

**/*

```
public class CounterViewModel extends ViewModel {
```

*/***

** LiveData המכיל את המונה ערך את המכיל LiveData*

**/*

```
private final MutableLiveData<Integer> counterValueLiveData;
```

*/***

** LiveData המכיל את המונה תיאור את המכיל LiveData*

**/*

```
private final MutableLiveData<String> counterStatusLiveData;
```

*/***

** המספרי הערך על הלוגיות הפעולות את המבצע המונה מנהל*

**/*

```
private final CounterManager counterManager;
```

*/***

** ViewModel-ה של בנאי*

** ההתחלתיים הנתונים את ומעדכן CounterManager-ה ואת LiveData-ה את מאתחל*

**/*

```
public CounterViewModel() {
```

```
    counterValueLiveData = new MutableLiveData<>();
```

```
    counterStatusLiveData = new MutableLiveData<>();
```

```
    counterManager = new CounterManager();
```

```
    updateCounterData();
```

```
}
```



```
/**
 * בלבד לקריאה המונה ערך של LiveData ה- את מחזירה *
 * @return LiveData אובייקט *
 */
public LiveData<Integer> getCounterValue() {
    return counterValueLiveData;
}

/**
 * בלבד לקריאה המונה סטטוס של LiveData ה- את מחזירה *
 * @return LiveData אובייקט *
 */
public LiveData<String> getCounterStatus() {
    return counterStatusLiveData;
}

/**
 * המונה להגדלת פקודה *
 * LiveData ה- של עדכון ומפעילה CounterManager ה- את מעדכנת *
 */
public void increaseCounter() {
    counterManager.increaseValue();
    updateCounterData();
}

/**
 * המונה להקטנת פקודה *

```

** LiveData ה של עדכון ומפעילה CounterManager ה את מעדכנת **

**/*

```
public void decreaseCounter() {
    counterManager.decreaseValue();
    updateCounterData();
}
```

*/***

** המונה לאיפוס פקודה **

** LiveData ה של עדכון ומפעילה CounterManager ה את מעדכנת **

**/*

```
public void resetCounter() {
    counterManager.resetValue();
    updateCounterData();
}
```

*/***

** CounterManager ב הנוכחי הערך סמך על LiveData ה את מעדכנת **

** (אפס/שלילי/חיובי) לסטטוס המתאים הטקסט את קובעת **

**/*

```
private void updateCounterData() {
    int currentValue = counterManager.getCurrentValue();

    counterValueLiveData.setValue(currentValue);

    if (currentValue > 0) {
        counterStatusLiveData.setValue("חיובי המספר");
    }
}
```

```

    } else if (currentValue < 0) {
        counterStatusLiveData.setValue("שליילי המספר");
    } else {
        counterStatusLiveData.setValue("אפס הוא המספר");
    }
}
}
}

```

מחלקת COUNTERMANAGER

```

/**
 * המונה של הלוגיקה ניהול על האחראית מחלקה *
 * המונה ערך את ולאפס להקטין, להגדיל מאפשרת המחלקה *
 * שבאפליקציה המודל מחלקת זו *
 */
public class CounterManager {
    /**
     * המונה של הנוכחי הערך *
     */
    private int counterValue;
    /**
     * 0-ל המונה את המאתחל CounterManager מחלקת של בנאי *
     */
    public CounterManager() {
        counterValue = 0;
    }
    /**
     * 1-ב המונה ערך את מגדיל *

```

```
*/
public void increaseValue() {
    counterValue++;
}
/**
 * 1-ב המונה ערך את מקטין *
 */
public void decreaseValue() {
    counterValue--;
}
/**
 * 0-ל המונה ערך את מאפס *
 */
public void resetValue() {
    counterValue = 0;
}
/**
 * המונה של הנוכחי הערך את מחזיר *
 * @return המונה של הנוכחי הערך *
 */
public int getCurrentValue() {
    return counterValue;
}
}
```

נספח ב אפליקציית קניות


```
public class MainActivity extends AppCompatActivity {  
    private EditText editTextProductName,editTextProductPrice;  
    private Button buttonAddProduct;  
    private TextView textViewItemCount,textViewTotalPrice;  
    private RecyclerView recyclerViewProducts;  
    private ShoppingAdapter shoppingAdapter;  
    private ShoppingViewModel shoppingViewModel;  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
  
        connectViews();  
        setupRecyclerView();  
        setupViewModel();  
        setupListeners();  
    }  
  
    private void connectViews() {  
        editTextProductName =findViewById(R.id.editTextProductName);  
        editTextProductPrice =findViewById(R.id.editTextProductPrice);  
        buttonAddProduct =findViewById(R.id.buttonAddProduct);  
        textViewItemCount =findViewById(R.id.textViewItemCount);  
        textViewTotalPrice =findViewById(R.id.textViewTotalPrice);  
        recyclerViewProducts =findViewById(R.id.recyclerViewProducts);  
    }  
}
```

```
private void setupRecyclerView() {

    shoppingAdapter = new ShoppingAdapter(
        new ArrayList<>(),
        shoppingItem -> shoppingViewModel.deleteProduct(shoppingItem)
    );

    recyclerViewProducts.setLayoutManager(
        new LinearLayoutManager(this)
    );
    recyclerViewProducts.setAdapter(shoppingAdapter);
}

private void setupViewModel() {

    shoppingViewModel = new ViewModelProvider(this).get(ShoppingViewModel.class);
    shoppingViewModel.getShoppingList().observe(this, shoppingItems -> {
        shoppingAdapter.setShoppingList(shoppingItems);

        textViewItemCount.setText("פריטים מספר: " + shoppingItems.size());
    });
    shoppingViewModel.getTotalPrice().observe(this, totalPrice -> {
        textViewTotalPrice.setText("לתשלום סה"כ: " + totalPrice + " ₪");
    });
    shoppingViewModel.getMessage().observe(this, message -> {
        if (message != null && !message.isEmpty()) {
```

```

Toast.makeText(MainActivity.this, message, Toast.LENGTH_SHORT).show();
shoppingViewModel.clearMessage();
    }
});
}
private void setupListeners() {
    buttonAddProduct.setOnClickListener(v -> {
        String productName = editTextProductName.getText().toString();
        String productPrice = editTextProductPrice.getText().toString();
        shoppingViewModel.addProduct(productName, productPrice);
        editTextProductName.setText("");
        editTextProductPrice.setText("");
    });
}
}
}

```

מחלקה המייצגת מוצר אחד ShoppingItem

```

public class ShoppingItem {
    private int id;
    private String productName;
    private int productPrice;
    public ShoppingItem() {
    }
    public ShoppingItem(String productName, int productPrice) {
        this.productName = productName;
        this.productPrice = productPrice;
    }
}

```

```
public ShoppingItem(int id, String productName, int productPrice) {
    this.id = id;
    this.productName = productName;
    this.productPrice = productPrice;
}

public int getId() {    return id;    }
public String getProductName() {    return productName;    }
public int getProductPrice() {    return productPrice;    }
public void setId(int id) {    this.id = id;    }
public void setProductName(String productName) {    this.productName = productName;    }
public void setProductPrice(int productPrice) {    this.productPrice = productPrice;    }
}
```

מחלקת ShoppingViewModel

```
public class ShoppingViewModel extends AndroidViewModel {
    private final MutableLiveData<List<ShoppingItem>> shoppingListLiveData;
    private final MutableLiveData<String> messageLiveData;
    private final MutableLiveData<Integer> totalPriceLiveData;

    private final ShoppingManager shoppingManager;

    public ShoppingViewModel(@NonNull Application application) {
        super(application);
    }
}
```

```

shoppingListLiveData = new MutableLiveData<>(new ArrayList<>());
messageLiveData = new MutableLiveData<>("");
totalPriceLiveData = new MutableLiveData<>(0);
shoppingManager = new ShoppingManager(application);
loadProducts();
}

public LiveData<List<ShoppingItem>> getShoppingList() { return shoppingListLiveData; }
public LiveData<String> getMessage() { return messageLiveData; }
public LiveData<Integer> getTotalPrice() { return totalPriceLiveData; }
public void loadProducts() {
    List<ShoppingItem> shoppingItems = shoppingManager.getAllProducts();
    shoppingListLiveData.setValue(shoppingItems);
    int totalPrice = shoppingManager.calculateTotalPrice();
    totalPriceLiveData.setValue(totalPrice);
}

public void addProduct(String productName, String productPrice) {
    if (productName == null || productName.trim().isEmpty()) {
        messageLiveData.setValue("מוצר שם להזין יש");
        return;
    }
    if (productPrice == null || productPrice.trim().isEmpty()) {
        messageLiveData.setValue("מחיר להזין יש");
        return;
    }
    int price;

    try {

```



```
price = Integer.parseInt(productPrice);
} catch (NumberFormatException e) {
    messageLiveData.setValue("ישלם מספר להיות חייב המחיר");
    return;
}
if (price <= 0) {
    messageLiveData.setValue("0-מ גדול להיות חייב המחיר");
    return;
}
ShoppingItem shoppingItem = new ShoppingItem(productName.trim(), price);
shoppingManager.insertProduct(shoppingItem);
loadProducts();
messageLiveData.setValue("בהצלחה נוסף המוצר");
}
public void deleteProduct(ShoppingItem shoppingItem) {
    if (shoppingItem == null) {
        messageLiveData.setValue("למחיקה מוצר נבחר לא");
        return;
    }
    shoppingManager.deleteProduct(shoppingItem);
    loadProducts();
    messageLiveData.setValue("בהצלחה נמחק המוצר");
}
public void clearMessage() { messageLiveData.setValue(""); }
}
```

מחלקת ShoppingManager

```
public class ShoppingManager extends SQLiteOpenHelper {
    private static final String DATABASE_NAME = "shopping_db";
    private static final int DATABASE_VERSION = 1;
    private static final String TABLE_PRODUCTS = "products";
    private static final String COLUMN_ID = "id";
    private static final String COLUMN_NAME = "product_name";
    private static final String COLUMN_PRICE = "product_price";

    public ShoppingManager(Context context) {
        super(context.getApplicationContext(), DATABASE_NAME, null, DATABASE_VERSION);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        String createTableQuery =
            "CREATE TABLE " + TABLE_PRODUCTS + " (" +
                COLUMN_ID + " INTEGER PRIMARY KEY AUTOINCREMENT, " +
                COLUMN_NAME + " TEXT NOT NULL, " +
                COLUMN_PRICE + " INTEGER NOT NULL" +
            ")";

        db.execSQL(createTableQuery);
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
```

```
db.execSQL("DROP TABLE IF EXISTS " + TABLE_PRODUCTS);
onCreate(db);
}

public void insertProduct(ShoppingItem shoppingItem) {
    SQLiteDatabase db = getWritableDatabase();
    ContentValues values = new ContentValues();
    values.put(COLUMN_NAME, shoppingItem.getProductName());
    values.put(COLUMN_PRICE, shoppingItem.getProductPrice());
    db.insert(TABLE_PRODUCTS, null, values);
    db.close();
}

public List<ShoppingItem> getAllProducts() {
    List<ShoppingItem> shoppingItems = new ArrayList<>();
    SQLiteDatabase db = getReadableDatabase();
    Cursor cursor = db.rawQuery(
        "SELECT * FROM " + TABLE_PRODUCTS + " ORDER BY " + COLUMN_ID + " DESC",
        null
    );
    if (cursor.moveToFirst()) {
        do {
            int id = cursor.getInt(
                cursor.getColumnIndexOrThrow(COLUMN_ID)
            );
            String productName = cursor.getString(
                cursor.getColumnIndexOrThrow(COLUMN_NAME)
            );
        } while (cursor.moveToNext());
    }
}
```

```
);
int productPrice = cursor.getInt(
    cursor.getColumnIndexOrThrow(COLUMN_PRICE)
);
ShoppingItem shoppingItem =
    new ShoppingItem(id, productName, productPrice);
shoppingItems.add(shoppingItem);
} while (cursor.moveToNext());
}
cursor.close();
db.close();
return shoppingItems;
}

public void deleteProduct(ShoppingItem shoppingItem) {
    if (shoppingItem == null) {
        return;
    }
    SQLiteDatabase db = getWritableDatabase();
    db.delete(
        TABLE_PRODUCTS,
        COLUMN_ID + "=?",
        new String[]{String.valueOf(shoppingItem.getId())}
    );
    db.close();
}

public int calculateTotalPrice() {
    int totalPrice = 0;
```

```

SQLiteDatabase db = getReadableDatabase();
Cursor cursor = db.rawQuery(
    "SELECT SUM(" + COLUMN_PRICE + ") FROM " + TABLE_PRODUCTS,
    null
);
if (cursor.moveToFirst()) {
    totalPrice = cursor.getInt(0);
}
cursor.close();
db.close();
return totalPrice;
}
}

```

מחלקת ShoppingAdapter

```

public class ShoppingAdapter extends RecyclerView.Adapter<ShoppingAdapter.ShoppingViewHolder> {
    public interface OnDeleteClickListener {
        void onDeleteClick(ShoppingItem shoppingItem);
    }
    private List<ShoppingItem> shoppingList;
    private final OnDeleteClickListener onDeleteClickListener;
    public ShoppingAdapter(List<ShoppingItem> shoppingList,
        OnDeleteClickListener onDeleteClickListener) {
        this.shoppingList = shoppingList;
        this.onDeleteClickListener = onDeleteClickListener;
    }
}

```

```

}
@NonNull
@Override
public ShoppingViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {

    View view = LayoutInflater.from(parent.getContext())
        .inflate(R.layout.item_product, parent, false);

    return new ShoppingViewHolder(view);
}
@Override
public void onBindViewHolder(@NonNull ShoppingViewHolder holder, int position) {

    ShoppingItem shoppingItem = shoppingList.get(position);
    holder.textViewProductName.setText(shoppingItem.getProductName());
    holder.textViewProductPrice.setText(    "מחיר: " + shoppingItem.getProductPrice() + " ₪    ");
    holder.buttonDeleteProduct.setOnClickListener(v -> {
        if (onDeleteClickListener != null) {
            onDeleteClickListener.onDeleteClick(shoppingItem);
        }
    });
}
@Override
public int getItemCount() {    return shoppingList.size();    }

public void setShoppingList(List<ShoppingItem> shoppingList) {
    this.shoppingList = shoppingList;
}

```



```
        notifyDataSetChanged();
    }

    public static class ShoppingViewHolder extends RecyclerView.ViewHolder {
        TextView textViewProductName;
        TextView textViewProductPrice;
        Button buttonDeleteProduct;
        public ShoppingViewHolder(@NonNull View itemView) {
            super(itemView);
            textViewProductName = itemView.findViewById(R.id.textViewProductName);
            textViewProductPrice = itemView.findViewById(R.id.textViewProductPrice);
            buttonDeleteProduct = itemView.findViewById(R.id.buttonDeleteProduct);
        }
    }
}
```